

AI TOOL USED :CODEX (BY chatgpt)

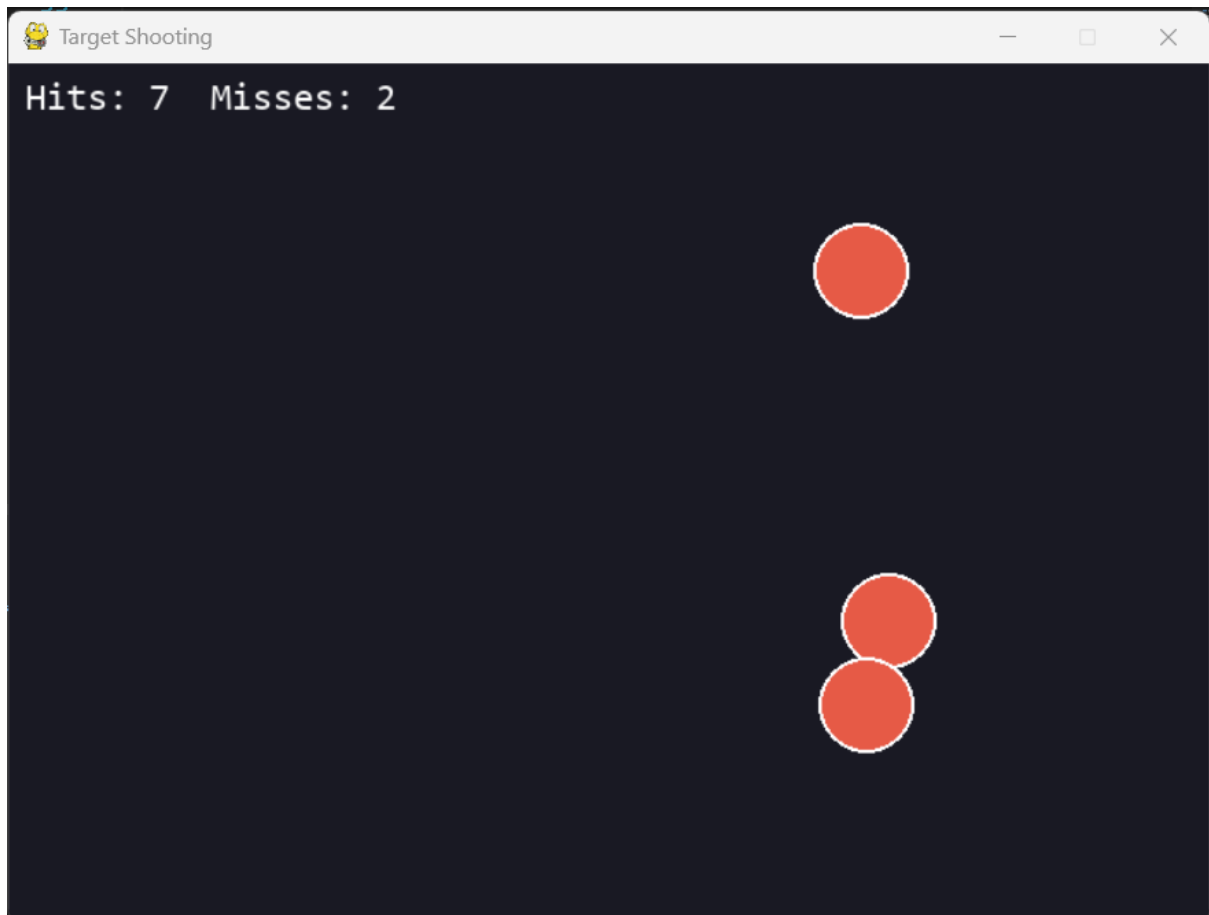
Task 1:

I am the gamer there is some issue with game that i have divided them into task please implement updated part of the code in differnt color so i differentiate later so the first task is

Task 1: Fix the hit-detection bug

A click is supposed to register as a hit only when it lands inside the visible circle of a target. In the current build, `Target.get_bounding_rect` (in `game/target.py`) builds its hitbox as `pygame.Rect(self.x, self.y, radius2, radius2)` - but `x, y` is the center of the circle, not its top-left corner. That shifts the entire hitbox down and to the right of where the target actually appears, so clicks near one edge of a target can miss it, while clicks clearly outside the other edge can register as a hit. Fix the hit test in `check_hit` (in `game/hit_detection.py`) so it's based on the target's real circular shape.

```
5
6 def check_hit(targets, click_pos):
7     """
8     Returns the target that was clicked, or None if the click missed
9     every target.
10    """
11    for target in targets:
12        # TASK 1 UPDATE: test the click's distance from the circle's center.
13        # This accepts only points inside the visible circular target.
14        offset_x = click_pos[0] - target.x
15        offset_y = click_pos[1] - target.y
16        if offset_x ** 2 + offset_y ** 2 <= target.radius ** 2:
17            return target
18    return None
```



Task 2: Implement moving targets

Make the targets move within the play area instead of sitting still, using at least two different speeds or movement patterns. Targets should bounce off the edges of the play area rather than leaving the screen, and clicking one accurately should still work while it's moving.

Prompt

The first implementation is working as expected, and now please do the second task.

Task 2: Implement moving targets

Make the targets move within the play area instead of sitting still, using at least two different speeds or movement patterns. Targets should bounce off the edges of the play area rather than leaving the screen, and clicking one accurately should still work while it's moving.

Please highlight updated or modified code.

Task 3: Implement combo scoring

Task 1 and Task 2 are implemented perfectly. Now move ahead with Task 3.

Add a score and combo multiplier. Each hit should add points based on the current combo multiplier, and consecutive hits with no misses in between should raise that multiplier so later hits are worth more. A miss should reset the combo. Display both the score and the current combo multiplier during play.

Please highlight updated or modified code.

Task 4: Implement a timed shooting round

Task 1 to Task 3 are implemented successfully. Please now implement Task 4.

Add a 30-second countdown for the round. Display the remaining time on screen. Once it reaches zero, no further shots should be accepted, the final score should be shown clearly, and a new round should be startable with the score, combo, and timer all reset.

Please highlight updated or modified code.

now all the task is done please check all the expected behaviour is implemented correctly

***** Expected Behavior****

\- Clicking squarely inside a visible target should always register as a hit; clicking clearly outside every target should always register as a miss - test this specifically near the edges of a target, not just dead center.

\- Targets move continuously and stay within the play area at all times.

\- Hits build a combo (more points per hit the longer the streak); a single miss brings the combo back down.

\- The round lasts exactly 30 seconds. Once time runs out, clicks should no longer do anything, and the final score should be shown clearly.